object listed. Calling `reverse()` with the right argument gives you the URL of the listing page, which you augment with a query parameter to filter the page.

Let's add a band-membership link to the Musician Listing page. You need a new callable for the column—I've named mine `show_bands`. Inside the callable, use `reverse()` to look up the URL for the Band Listing page. This page is named `admin:bands_band_changelist` (`bands` for the app and `band` for the `Model`). To filter the listing by a musician's bands, you need the IDs of those bands. Query those IDs, and then construct an `?id__in` query parameter to append to your listing page URL.

Recall from the chapter on templates that Django escapes HTML, by default. You can't just return text with an `<a>` tag inside it; you need to mark it as safe for rendering. The `format_html()` function from the `django.utils.html` module lets you build safe HTML. This function uses paired braces ({}) as placeholders, allowing you to parameterize a string with an HTML link inside it. Like with `show_weekday`, add the `.short_description` attribute to change the column title. Make the changes shown in the following listing to your admin.py file, and then visit the Django Admin to see your new content.

Listing 5.6 The `MusicianAdmin` class with links to the `Band` listing page

```
# RiffMates/bands/admin.py
...
from django.utils.html import format_html
from django.urls import reverse
...
@admin.register(Musician)
class MusicianAdmin(admin.ModelAdmin):
    list_display = ("id", "last_name", "first_name", "birth", "show_weekday",
        "show_bands")
    ...

    def show_bands(self, obj):
        bands = obj.band_set.all()
        if len(bands) == 0:
            return format_html("<i>None</i>")

        plural = ""
        if len(bands) > 1:
            plural = "s"

        parm = "?id__in=" + ",".join([str(b.id) for b in bands])
        url = reverse("admin:bands_band_changelist") + parm
        return format_html('<a href="{}">Band{}</a>', url, plural)
    show_bands.short_description = "Bands"
```

Import format_html and reverse for use in the method.

Add the show_bands callable to the columns.

Query all bands associated with this musician.

If there are no bands, use format_html to show an italicized None.

If there is more than one band, the output should say Bands.

Look up the Band listing page, and then add the query parameter to the end.

Query parameter using the __in suffix and a comma-separated list of Band IDs.

Create a safe <a> tag using the URL and the pluralization of Band.

Start your development server, and then return to the Musician Listing page. You now have a Bands column with clickable links. Each link goes to a filtered version of the Band Listing page, showing only those bands the musician is a member of.